

many disadvantages of tightly coupled technologies, a higher maintenance cost being the most common. Another disadvantage is when one application calls another application through RPC, the other application must be available to receive the call or else the whole architecture fails. Figure 1 shows the architecture of two tightly coupled distributed applications.

To overcome the problems related to tightly coupled RPC, MOM (Message-Oriented Middleware) is used where one application sends messages to MOM and, on the other side, the other application receives the messages from MOM. Loosely coupled applications have greater advantages compared to tightly coupled ones when considering distributed applications. Figure 2 shows the architecture of two loosely coupled distributed applications.

ActiveMQ is used to overcome the disadvantages of RPC based tightly coupled distributed applications. The application relies on ActiveMQ and there is a guaranteed delivery of messages. ActiveMQ acts as a middleman between distributed applications and interacts in an asynchronous manner.

Installation of ActiveMQ

To install ActiveMQ, download the application from <http://activemq.apache.org/download-archives.html>. Extract and run the activemq script from the bin directory to start the ActiveMQ server as shown in Figure 3. Next, check if <http://localhost:8161> is accessible from the browser. The ActiveMQ web console shown in Figure 4 is used to monitor ActiveMQ.

Here's an example of how ActiveMQ is used.

ActiveMQ uses a notation called *Producer/Publisher* and *Consumer/Subscriber*. *Producer* produces the messages and *Consumer* consumes it. A *Producer* can produce messages both on queues (messages have to be delivered to only one consumer) and topics (a distributed mechanism for publishing multiple messages that are delivered to multiple consumers).

Application 1 (Consumer): To connect to the ActiveMQ server, ActiveMQ client libraries have to be added. Create a Maven project as shown in Figure 5 and add the Maven dependency in pom.xml as follows:

```
<dependency>
  <groupId>org.apache.activemq</groupId>
  <artifactId>activemq-core</artifactId>
  <version>5.7.0</version>
</dependency>
```

```
Consumer Class -
import javax.jms.Connection;
import javax.jms.Destination;
import javax.jms.Message;
import javax.jms.MessageConsumer;
import javax.jms.Session;
import javax.jms.TextMessage;
```

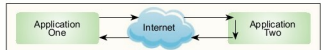


Figure 1: Tightly coupled distributed applications

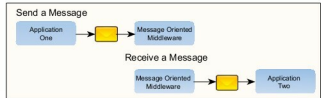


Figure 2: Loosely coupled distributed applications

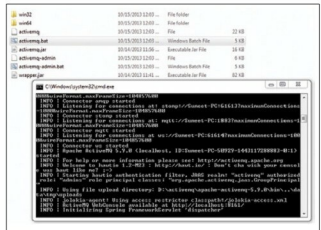


Figure 3: ActiveMQ server running



Figure 4: ActiveMQ WebConsole

```
import org.apache.activemq.ActiveMQConnectionFactory;
```

```
public class Consumer implements Runnable {

    public static void main(String[] args) throws
    InterruptedException {
        Consumer consumer = new Consumer();
        Thread t1 = new Thread(consumer);
        t1.start();
    }

    public void run() {
        while (true) {
```